
MAINCRAFTS TECHNOLOGY

Introduction to Embedded Systems & IoT Device Design

Embedded Systems & Internet of Things (IoT) Task- 1

Project

Smart Temperature & Humidity Monitoring System

Submitted by

Shree Harshan K

Date: 26.06.2026

Table of Contents

Table of Contents	0
1. Introduction to Embedded Systems	1
1.1 Characteristics of Embedded Systems	1
1.2 Real-World Applications	1
1.3 Microcontroller vs Microprocessor	1
2. Component Study	2
2.1 Microcontrollers	2
Arduino UNO	2
ESP32	2
Raspberry Pi Pico	2
2.2 Sensors	2
Temperature Sensor (DHT22)	2
Motion Sensor (PIR)	2
Light Sensor (LDR)	2
3. Project Details	3
3.1 Project Title	3
3.2 Objective	3
3.3 Components Used	3
3.4 Circuit Diagram	3
3.5 Pin Configurations	4
4. Working Principle	5
4.1 System Workflow	5
5. Source Code with Explanation	6
5.1 Code Walkthrough	7
6. Results	8
6.1 Normal Operating Condition	8
6.2 Alert Condition	8
7. Real-World Applications	9
Greenhouses and Agriculture	9
Server Rooms and Data Centres	9
Food Storage and Cold Chain Logistics	9
Laboratories and Healthcare Facilities	9
Smart Home Automation	9
Industrial Safety Systems	9
8. Conclusion	10

1. Introduction to Embedded Systems

An embedded system is an electronic system built around a microcontroller or microprocessor that is designed to perform one or a few dedicated functions, often under real-time constraints. Unlike a general-purpose computer, which can run many different programs, an embedded system is tightly coupled to its hardware and optimised for a specific job.

These systems are found everywhere in modern life. From the anti-lock braking system in your car to the timer in a microwave oven, embedded systems silently run the devices we depend on every day.

1.1 Characteristics of Embedded Systems

- Dedicated functionality – designed to perform a specific task
- Real-time operation – must respond to inputs within strict time limits
- Low power consumption – often battery-operated or energy-constrained
- Compact size – integrated onto a single board or chip
- High reliability – expected to run without failure for extended periods

1.2 Real-World Applications

Embedded systems are the backbone of the modern technological world. Some of the sectors where they are heavily used:

- Smart Home Devices – smart thermostats, automated lighting, security cameras
- Automotive Systems – engine control units, airbag controllers, GPS navigation
- Healthcare Equipment – pacemakers, blood glucose monitors, infusion pumps
- Industrial Automation – conveyor control, PLC-based manufacturing lines
- Robotics – motor controllers, sensor fusion systems, autonomous navigation
- Consumer Electronics – smart televisions, washing machines, digital cameras

1.3 Microcontroller vs Microprocessor

A common source of confusion is the difference between a microcontroller and a microprocessor. The table below clarifies the key distinctions:

Microcontroller	Microprocessor
Architecture: CPU, RAM, ROM, and I/O peripherals all integrated on a single chip. Self-contained and ready to use.	Architecture: Only the CPU core. Requires external RAM, ROM, and interface chips to function.
Power Consumption: Microcontrollers consume much less power – ideal for battery-operated devices.	Power Consumption: Microprocessors consume huge power – ideal for high end usage devices.
Cost: Microcontrollers are significantly cheaper due to higher integration.	Cost: Microprocessors are highly priced due to its additional components

2. Component Study

2.1 Microcontrollers

Arduino UNO

The Arduino UNO is one of the most popular beginner-friendly microcontroller boards, based on the ATmega328P. It has 14 digital I/O pins, 6 analogue input pins, and runs at 16 MHz. It is widely used for prototyping because of its large community, extensive library support, and easy programming via the Arduino IDE.

ESP32

The ESP32 is a more powerful microcontroller that includes a dual-core processor running at up to 240 MHz, along with built-in Wi-Fi and Bluetooth. This makes it ideal for IoT projects where wireless communication is required. It also has more GPIO pins and supports higher-speed interfaces like SPI and I2C. The ESP32 was chosen for this project because of its capability to handle multiple tasks simultaneously and its strong community support.

Raspberry Pi Pico

The Raspberry Pi Pico is based on the RP2040 microcontroller chip, designed by the Raspberry Pi Foundation. It features a dual-core ARM Cortex-M0+ processor running at 133 MHz. Unlike the Raspberry Pi single-board computer, the Pico is a true microcontroller board suited for embedded systems work. It supports MicroPython and C/C++, making it accessible to both beginners and experienced developers.

2.2 Sensors

Temperature Sensor (DHT22)

The DHT22 is a digital temperature and humidity sensor that provides calibrated readings over a single-wire serial interface. It measures temperatures from -40°C to $+80^{\circ}\text{C}$ and relative humidity from 0% to 100% RH. Compared to the DHT11, it offers better accuracy and a wider measurement range, making it the preferred choice for this project.

Motion Sensor (PIR)

A Passive Infrared (PIR) sensor detects infrared radiation emitted by warm bodies such as humans or animals. When movement is detected within its field of view, it outputs a HIGH signal to the microcontroller. PIR sensors are commonly used in security alarms, automatic lighting, and occupancy detection systems.

Light Sensor (LDR)

A Light Dependent Resistor (LDR), also called a photoresistor, changes its resistance based on the intensity of incident light. In bright conditions, its resistance drops significantly; in darkness, resistance increases. LDRs are used in automatic street lights, camera exposure control, and brightness adjustment circuits.

3. Project Details

3.1 Project Title

Smart Temperature and Humidity Monitoring System

3.2 Objective

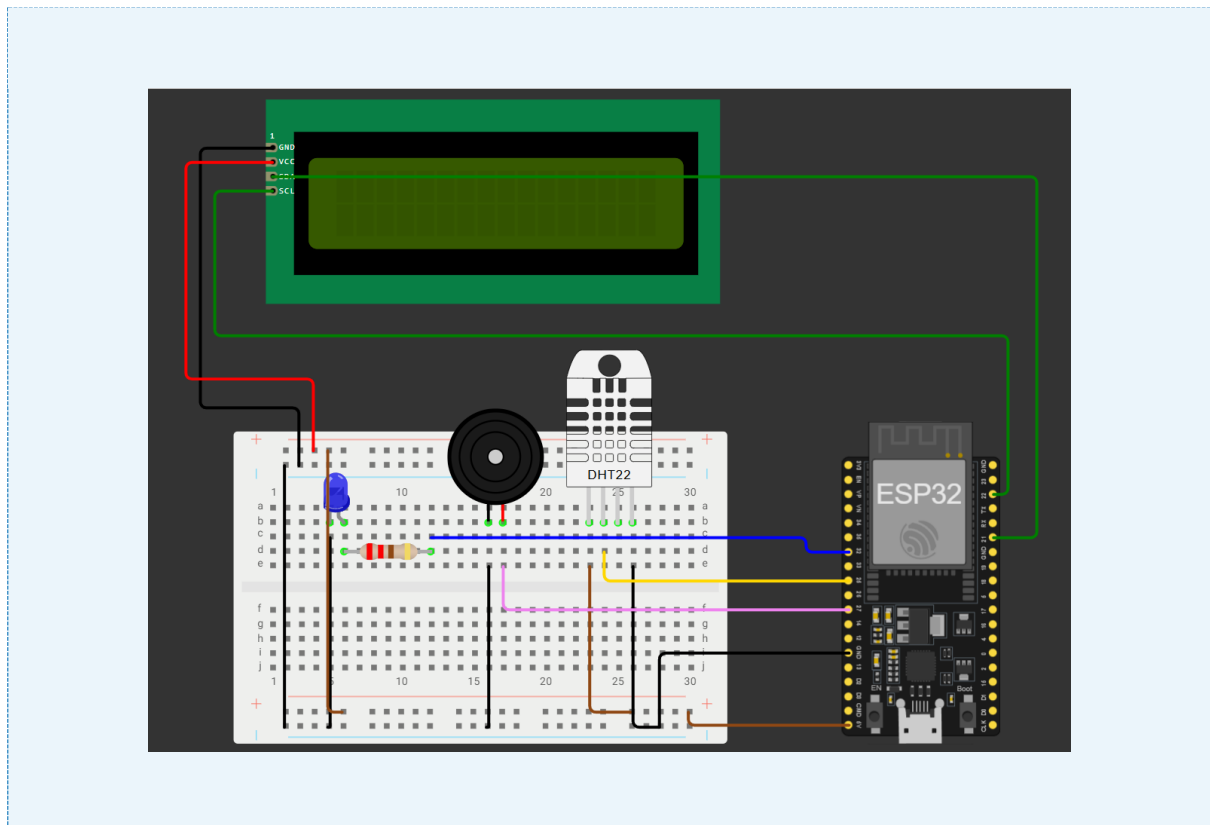
To design and simulate an IoT-based monitoring system that continuously reads temperature and humidity values from a DHT22 sensor, displays the readings on a 16×2 I2C LCD, and activates a visual (LED) and audible (buzzer) alert whenever the readings cross predefined safety thresholds.

3.3 Components Used

- ESP32 – central microcontroller
- DHT22 – temperature and humidity sensor
- 16×2 I2C LCD Display – shows real-time readings
- LED – visual indicator for alert state
- Buzzer – audible alert for threshold breaches
- Wokwi ESP32 Simulator – simulation environment

3.4 Circuit Diagram

The circuit was designed and tested in the Wokwi ESP32 online simulator. The diagram below shows all component connections.



3.5 Pin Configurations

Component	ESP32 Pin
DHT22 – Data	GPIO 25
DHT22 – VCC	5V
DHT22 – GND	GND
LCD – SDA	GPIO 21
LCD – SCL	GPIO 22
LCD – VCC	5V
LCD – GND	GND
LED	GPIO 32
Buzzer	GPIO 27

4. Working Principle

The system starts by initialising the I2C LCD display and the DHT22 sensor. Once initialised, the ESP32 enters a continuous loop where it reads the current temperature and humidity values from the DHT22 every two seconds.

These values are immediately printed on the 16×2 LCD screen so anyone nearby can check the environmental conditions at a glance. Simultaneously, the ESP32 compares the readings against two predefined threshold values:

Parameter	Threshold
Temperature	35.0°C
Humidity	80.0% RH

If either value exceeds its threshold, the system immediately switches into alert mode. In alert mode the LED on GPIO 32 is turned on, the buzzer on GPIO 27 generates a repeating beep pattern, and the LCD switches from displaying the numerical readings to showing a warning message. When the readings return to within the safe range, the system automatically deactivates the alert and resumes normal display mode.

During my testing in Wokwi, I manually adjusted the virtual sensor sliders to values above and below the thresholds to verify that the system responded correctly in both directions.

4.1 System Workflow

- DHT22 measures temperature and humidity
- ESP32 reads the sensor values every 2 seconds
- LCD displays the real-time readings
- ESP32 compares readings against thresholds
- If either threshold is breached:
 - LED turns ON (GPIO 32)
 - Buzzer beeps repeatedly (GPIO 27)
 - LCD shows a warning message
- If readings are within safe range:
 - LED remains OFF
 - Buzzer remains silent
 - LCD shows temperature and humidity values

5. Source Code with Explanation

The following Arduino/C++ code was written for the ESP32. Each section is commented to explain its purpose.

```
// — Library Includes


---


#include <DHT.h>      // DHT22 temperature & humidity sensor library
#include <Wire.h>      // I2C communication library
#include <LiquidCrystal_I2C.h> // I2C LCD library

// — Pin Definitions


---


#define DHT_PIN  25  // DHT22 data line connected to GPIO 25
#define LED_PIN  32  // LED connected to GPIO 32
#define BUZZER_PIN 27 // Buzzer connected to GPIO 27
#define DHT_TYPE  DHT22 // Sensor type: DHT22

// — Threshold Values


---


#define TEMP_THRESHOLD 35.0 // Alert if temperature exceeds 35 °C
#define HUM_THRESHOLD  80.0 // Alert if humidity exceeds 80 %

// — Object Instantiation


---


DHT dht(DHT_PIN, DHT_TYPE);      // Create DHT sensor object
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD at I2C address 0x27, 16 cols, 2 rows

// — setup() – runs once on power-up


---


void setup() {
  Serial.begin(115200); // Start serial monitor at 115200 baud

  dht.begin();          // Initialise the DHT22 sensor

  lcd.init();           // Initialise the LCD
  lcd.backlight();      // Turn on the LCD backlight
  lcd.print("System Ready"); // Show startup message
  delay(2000);          // Display for 2 seconds
  lcd.clear();

  pinMode(LED_PIN, OUTPUT); // Set LED pin as output
  pinMode(BUZZER_PIN, OUTPUT); // Set buzzer pin as output

  // Ensure both are off at startup
  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);
}

// — loop() – runs repeatedly


---


void loop() {
  float temperature = dht.readTemperature(); // Read temperature in Celsius
  float humidity    = dht.readHumidity();    // Read relative humidity (%)

  // Validate sensor readings before using them
  if (isnan(temperature) || isnan(humidity)) {
    lcd.clear();
    lcd.print("Sensor Error!");
    delay(2000);
    return; // Skip this loop iteration
  }
}
```

```
// Display real-time readings on LCD
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Temp: ");
lcd.print(temperature, 1); // 1 decimal place
lcd.print(" C");
lcd.setCursor(0, 1);
lcd.print("Hum: ");
lcd.print(humidity, 1);
lcd.print(" %");

// Check thresholds and trigger alert if needed
if (temperature > TEMP_THRESHOLD || humidity > HUM_THRESHOLD) {
  // — ALERT MODE

  digitalWrite(LED_PIN, HIGH); // LED ON

  // Buzzer: two short beeps
  digitalWrite(BUZZER_PIN, HIGH); delay(200);
  digitalWrite(BUZZER_PIN, LOW); delay(200);
  digitalWrite(BUZZER_PIN, HIGH); delay(200);
  digitalWrite(BUZZER_PIN, LOW);

  // Show alert message on second row
  lcd.setCursor(0, 1);
  lcd.print("!! ALERT !! ");
} else {
  // — NORMAL MODE

  digitalWrite(LED_PIN, LOW); // LED OFF
  digitalWrite(BUZZER_PIN, LOW); // Buzzer OFF
}

delay(2000); // Wait 2 seconds before next sensor read
}
```

5.1 Code Walkthrough

The code begins by including three libraries: DHT.h for sensor communication, Wire.h for I2C protocol support, and LiquidCrystal_I2C.h for driving the LCD. All pin numbers and threshold values are defined as constants at the top so they can be easily changed without searching through the code.

The setup() function runs once when the board powers on. It initialises the serial monitor, the DHT22 sensor, and the LCD. Both the LED and buzzer pins are configured as outputs and set to LOW to ensure nothing is accidentally activated at startup.

The loop() function runs indefinitely. Each iteration reads fresh temperature and humidity values, validates them to catch any sensor read errors, then updates the LCD. If either threshold is exceeded, the alert system is activated. If both readings are within the safe range, all outputs are kept off.

6. Results

The system was tested by running the simulation in Wokwi and manually adjusting the DHT22 virtual sensor sliders to different temperature and humidity values. The following results were observed:

6.1 Normal Operating Condition

- Temperature and humidity readings were displayed correctly on the LCD.
- LED remained OFF.
- Buzzer remained silent.
- System updated every 2 seconds as expected.

6.2 Alert Condition

- When temperature was set above 35°C, the alert system activated immediately.
- When humidity was set above 80%, the alert system also activated.
- LED turned ON during alert conditions.
- Buzzer generated a repeating double-beep pattern.
- LCD displayed the ALERT message on the second row.
- System automatically returned to normal mode when values dropped below thresholds.

Test Condition	Result
Temp = 28°C, Hum = 60%	Normal – LCD shows values, LED OFF, Buzzer OFF
Temp = 38°C, Hum = 60%	ALERT – LED ON, Buzzer beeps, LCD shows alert
Temp = 28°C, Hum = 85%	ALERT – LED ON, Buzzer beeps, LCD shows alert
Temp = 38°C, Hum = 85%	ALERT – LED ON, Buzzer beeps, LCD shows alert
Temp returned to 30°C, Hum 65%	Normal mode restored automatically

7. Real-World Applications

A system like this has several practical use-cases in everyday life and industry:

Greenhouses and Agriculture

Plants require precise temperature and humidity ranges to grow well. An automated monitoring system can alert farmers when conditions deviate from the optimal range, potentially saving entire crops from damage.

Server Rooms and Data Centres

Electronic equipment is highly sensitive to heat and moisture. Continuous monitoring with automatic alerts helps prevent hardware damage and costly downtime caused by overheating or condensation.

Food Storage and Cold Chain Logistics

Warehouses storing perishable goods need to maintain specific environmental conditions. A monitoring system ensures that temperature and humidity stay within food-safe limits and raises an alert if they do not.

Laboratories and Healthcare Facilities

Many laboratory experiments and pharmaceutical products require controlled environments. Automated monitoring reduces human error and provides a continuous record of environmental conditions.

Smart Home Automation

This system can be extended to control smart home devices such as air conditioners and dehumidifiers automatically based on sensor readings, improving comfort and energy efficiency.

Industrial Safety Systems

Manufacturing environments that handle chemicals or temperature-sensitive materials benefit from continuous environmental monitoring that can trigger shutdown procedures before unsafe conditions escalate.

8. Conclusion

This task gave me a solid introduction to embedded systems design and IoT concepts. Through research I learned about the key differences between microcontrollers and microprocessors, the roles of common sensors, and the fundamentals of what makes a system 'embedded'.

The hands-on simulation work was the most valuable part of the task. By building the Smart Temperature and Humidity Monitoring System in Wokwi, I was able to see how all the pieces fit together — from reading a sensor, to processing the data in code, to controlling output devices based on logic decisions.

The final simulated system met all the requirements: it displayed real-time environmental data on the LCD, activated both the LED and the buzzer when thresholds were exceeded, and automatically returned to normal operation when the environment was safe again.

This project has built my confidence in working with ESP32, Arduino C++, and hardware simulation tools, and I look forward to working on more complex embedded systems projects in the future.

Submitted by: Shree Harshan K

Task: Embedded Systems & Internet of Things (IOT) Task- 1 – Maincrafts Technology

Date: 29.06.2026